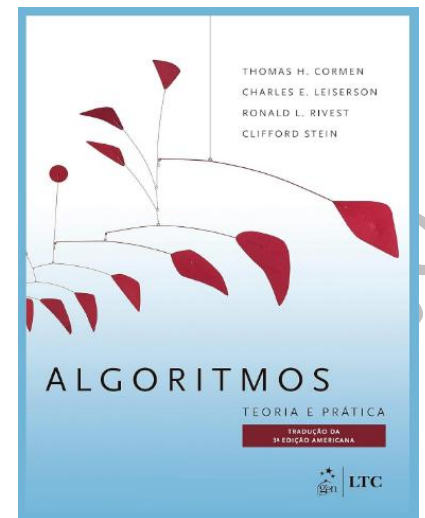


## Referência

### Algoritmos: Teoria e Prática

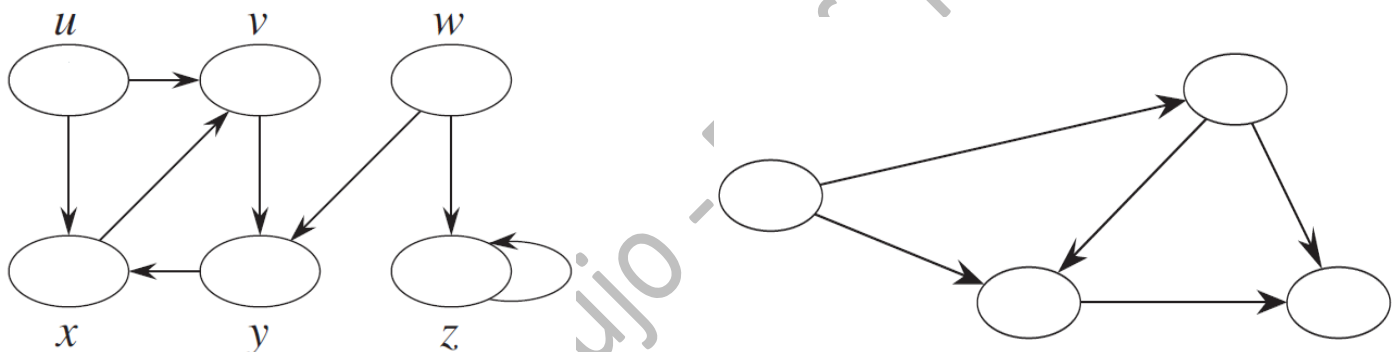
(3a. edição) por Thomas E. Cormen et al. LTC, 2012



## Ordenação Topológica

Um **dag** (“directed acyclic graph” – grafo orientado acíclico) é um grafo orientado que não possui nenhum circuito orientado.

**Exercício:** Os grafos abaixo são dags? Justifique.



O algoritmo de busca em profundidade pode ser utilizado para realizar uma ordenação topológica em um dag. Uma ordenação topológica em um dag é uma ordenação sequencial de todos os vértices do dag de tal forma que:

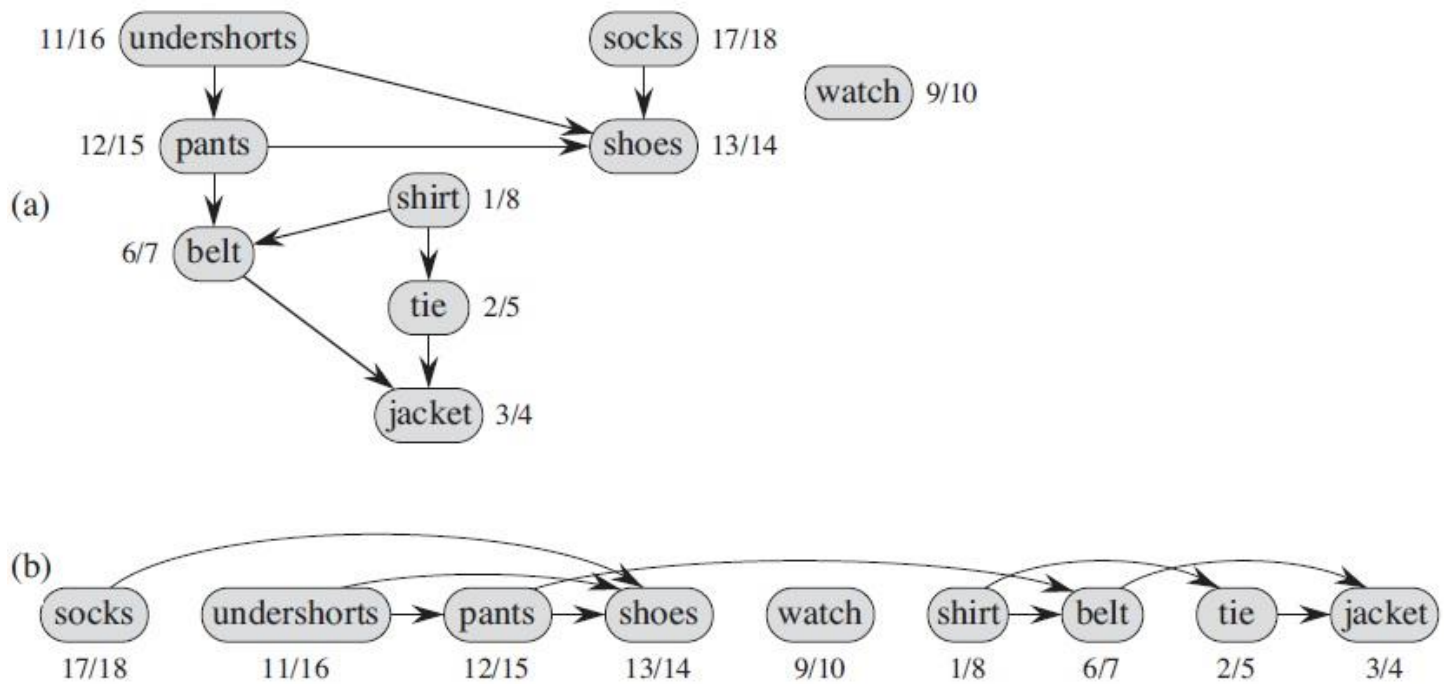
se o grafo  $G$  tiver um arco  $(u,v)$  então  $u$  deve aparecer antes de  $v$  na ordenação sequencial.

**Exercício:** Obtenha uma ordenação topológica dos grafos acima que forem dags.

Abaixo, temos um algoritmo simples que usa a busca em profundidade para fazer uma ordenação topológica:

### TOPOLOGICAL-SORT( $G$ )

- 1 call DFS( $G$ ) to compute finishing times  $v.f$  for each vertex  $v$
- 2 as each vertex is finished, insert it onto the front of a linked list
- 3 **return** the linked list of vertices

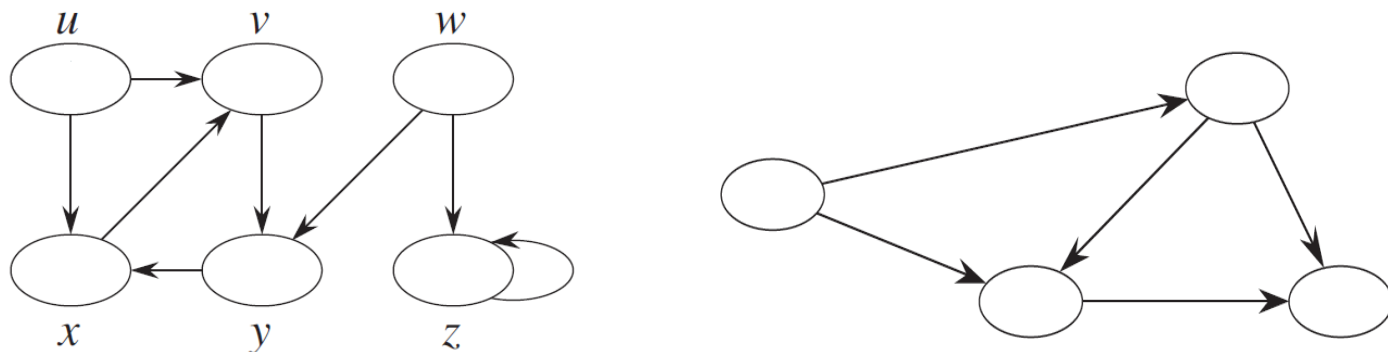


**Figure 22.7** (a) Professor Bumstead topologically sorts his clothing when getting dressed. Each directed edge  $(u, v)$  means that garment  $u$  must be put on before garment  $v$ . The discovery and finishing times from a depth-first search are shown next to each vertex. (b) The same graph shown topologically sorted, with its vertices arranged from left to right in order of decreasing finishing time. All directed edges go from left to right.

## Componentes Fortemente Conexas

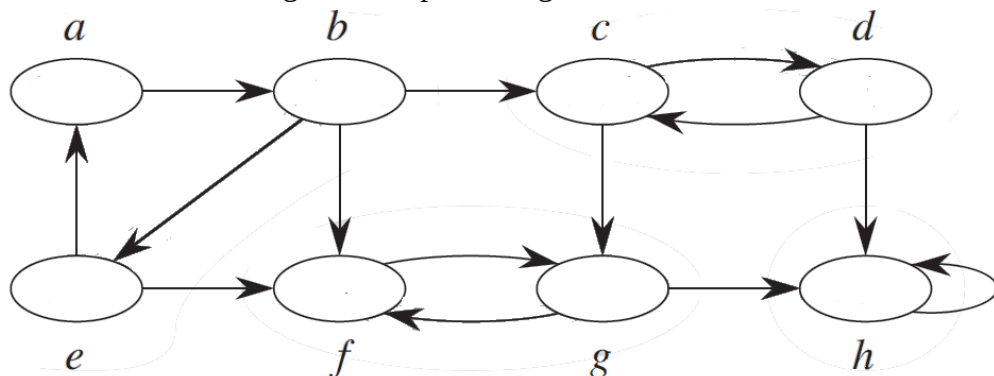
Um **componente fortemente conexo** de um grafo orientado  $G=(V,A)$  é um subgrafo maximal de  $G$  que é induzido por um subconjunto  $C \subseteq V$  com a seguinte propriedade  
para todo par  $u, v$  de vértices de  $C$ , existe um caminho orientado de  $u$  para  $v$  e também de  $v$  para  $u$ .

**Exercício:** Apresente componentes fortemente conexos dos grafos abaixo.



O **grafo transposto** (ou **reverso**) de um grafo orientado  $G=(V, A)$  é um grafo denotado por  $G^T = (V, A^T)$  tal que seu conjunto de vértices é igual ao de  $G$  e seus arcos são definidos como  $A^T = \{(u,v) / (v,u) \in A\}$ . Ou seja,  $A^T$  tem os mesmos arcos de  $A$ , só que no sentido contrário.

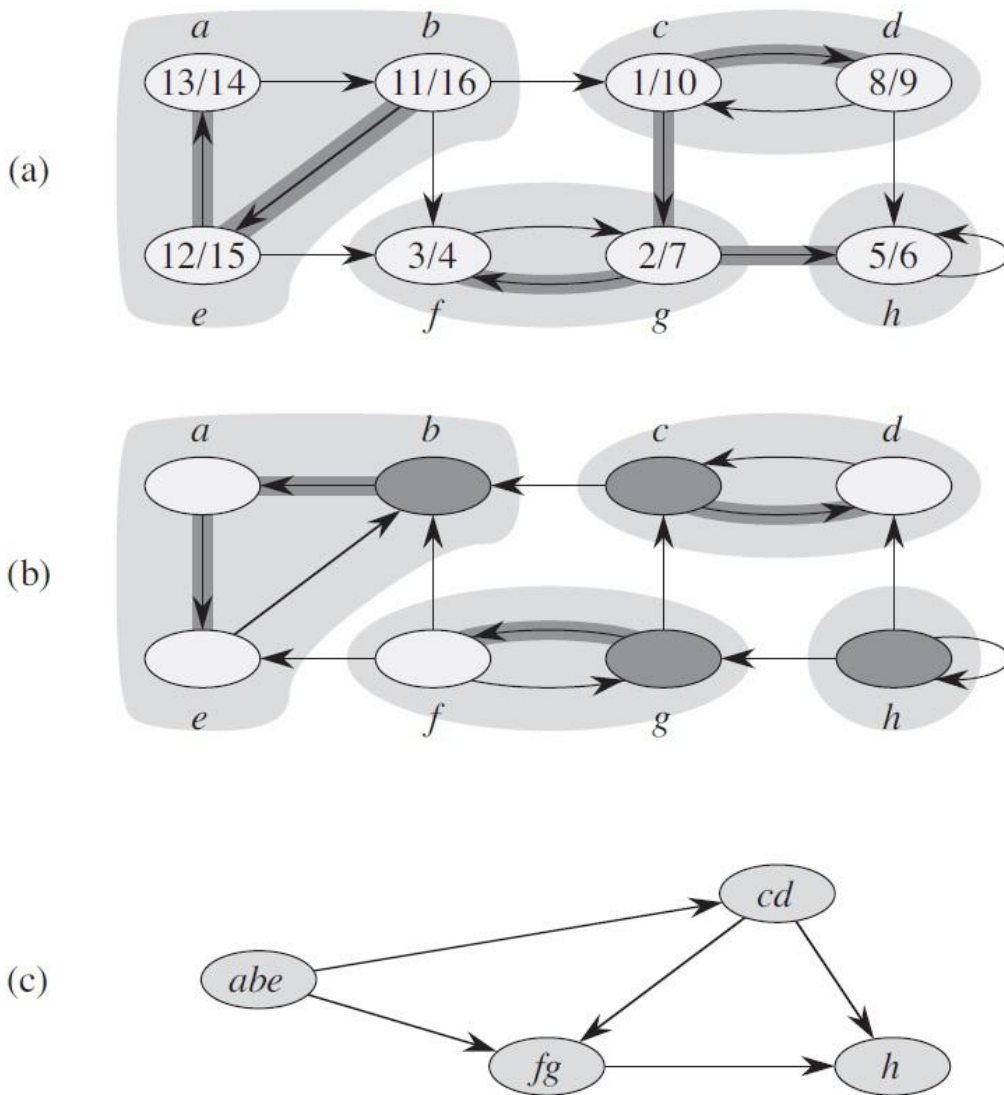
**Exercício:** Obtenha o grafo transposto do grafo abaixo:



Usando o algoritmo de busca em profundidade e o grafo transposto, podemos resolver o problema de obter os componentes fortemente conexos de um grafo  $G$ :

### STRONGLY-CONNECTED-COMPONENTS ( $G$ )

- 1 call  $\text{DFS}(G)$  to compute finishing times  $u.f$  for each vertex  $u$
- 2 compute  $G^T$
- 3 call  $\text{DFS}(G^T)$ , but in the main loop of DFS, consider the vertices in order of decreasing  $u.f$  (as computed in line 1)
- 4 output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component



**Figure 22.9** (a) A directed graph  $G$ . Each shaded region is a strongly connected component of  $G$ . Each vertex is labeled with its discovery and finishing times in a depth-first search, and tree edges are shaded. (b) The graph  $G^T$ , the transpose of  $G$ , with the depth-first forest computed in line 3 of STRONGLY-CONNECTED-COMPONENTS shown and tree edges shaded. Each strongly connected component corresponds to one depth-first tree. Vertices  $b$ ,  $c$ ,  $g$ , and  $h$ , which are heavily shaded, are the roots of the depth-first trees produced by the depth-first search of  $G^T$ . (c) The acyclic component graph  $G^{\text{SCC}}$  obtained by contracting all edges within each strongly connected component of  $G$  so that only a single vertex remains in each component.